UrbanFlow

Données, prototype local et services F1–F5

Master 2 SLCD — Big Data et NoSQL

7 novembre 2026

1 Périmètre et organisation du travail

UrbanFlow analyse les enregistrements de trajets des taxis jaunes de New York. Le jeu initial couvre janvier, février et mars 2024, dont le changement d'heure du 10 mars. Avril est réservé à la mise à jour F6 ; mai et juin complètent les six mois prévus pour les expériences. Brooklyn sert de point d'appui aux exemples, mais les données de toute la ville sont conservées. Les trajets enregistrés ne permettent pas de mesurer une demande non satisfaite ni l'ensemble des déplacements new-yorkais : les résultats portent sur cette archive de taxis [1].

La répartition retenue dans le binôme est la suivante : cette contribution porte sur la récupération des données, leur qualité, le modèle initial, l'API locale et les services F1 à F5 inclus. Le second membre prend en charge F6, les déploiements distribués, les expériences E1–E3 et leur analyse. Chacun rédige la partie correspondante du rapport ; les deux membres doivent pouvoir expliquer l'ensemble du projet. Codex a été utilisé pour développer et expliquer du code, préparer des vérifications et rédiger cette contribution. Les résultats chiffrés présentés ici proviennent des fichiers et mesures conservés dans le dépôt.

Le texte décrit l'état intégré de `main` après récupération du commit `5f11cfd`, et tient compte des reprises du second membre : identité des trajets, traçabilité détaillée, index de déploiement et optimisations de F3/F5. Les mesures du prototype du 5 novembre sont identifiées séparément ; elles ne valident pas à elles seules les changements ultérieurs [3, 5].

2 Données et règles de qualité

Les sources sont les Parquet mensuels *Yellow Taxi Trip Records* et le référentiel officiel des zones TLC. Le manifeste donne pour chaque fichier son URL, sa date de récupération, sa taille et son SHA-256. Les trois fichiers initiaux ont été récupérés le 2 novembre 2026 selon ce manifeste ; les empreintes complètes figurent en annexe A [2, 4].

Fichier 2024	Lignes sources	Profil activity	Profil flows	Profil journey
Janvier	2 964 624	2 929 120	2 898 055	2 841 251
Février	3 007 526	2 971 445	2 942 952	2 878 111
Mars	3 582 628	3 538 317	3 503 542	3 420 628

Ces effectifs sont ceux des bilans par fichier : au total, 9 554 778 lignes sources, pas un nombre garanti de déplacements physiques distincts. Ils ne sont pas non plus un nouveau comptage du déploiement intégré après déduplication entre fichiers. Le référentiel contient 265 zones. Dans le fichier de janvier, 25 258 lignes ont une zone de départ rattachée à Brooklyn ; Manhattan en compte 2 646 948. Le focus géographique ne doit donc pas masquer le déséquilibre des volumes [4].

Une ligne appartient à une période selon sa date réelle de départ, et non selon le mois du fichier. F1 et F3 en mode arrivées utilisent toutefois la zone *et* la date d'arrivée. Les intervalles sont semi-ouverts $[d_0, d_1[$ en heure locale `America/New_York`. Cette convention évite de compter deux fois une borne.

Les anomalies de qualité ne provoquent pas de suppression à l'import : elles sont conservées dans `quality_flags`, puis chaque analyse applique son profil. `activity` exclut les montants totaux négatifs et les doublons exacts ; `flows` ajoute les zones 264/265, sans localisation précise ; `journey` ajoute les durées nulles ou négatives, celles au-delà de six heures, les distances nulles ou supérieures à 100 miles, les vitesses supérieures à 80 miles/h et les heures DST non résolues. Ces seuils larges écartent des anomalies manifestes, sans définir un trajet « normal ». En janvier, le profil journey écarte 123 373 lignes, soit 4,16 %, contre 1,20 % pour activity. Les motifs peuvent se cumuler ; leur somme n'est pas le total des exclusions.

Un doublon exact est une copie de toutes les colonnes dans un même fichier ; seule la première occurrence reste exploitable. Une annulation est repérée par le fournisseur, les deux horodatages, les zones et un montant

opposé ; l'original positif est conservé. La détection est faite sur le fichier entier avant l'écriture par lots de 5 000, ce qui évite de dépendre des frontières de lots, mais impose un coût mémoire à mesurer à plus grand volume. Les anomalies ne sont pas toutes couvertes : le contrôle des zones vise notamment 264/265 et la détection des annulations reste interne à chaque fichier. L'annexe B documente huit anomalies ou particularités effectivement présentes avec leurs lignes sources.

3 Modèle local et accès à MongoDB

MongoDB fonctionne comme un serveur séparé dans Docker Compose ; l'application y accède par une URI réseau. Le déploiement local utilise MongoDB 8.0.16 sur `127.0.0.1:27018`, avec un volume persistant. L'ingestion et les contrôles Parquet sont en Python. L'API est en Spring Boot 3.5.7, Java 21, avec Spring Data MongoDB et `MongoTemplate`. JPA n'est pas utilisé : les requêtes sont des filtres et pipelines BSON explicites. Les contrôleurs valident les entrées, les services appliquent les règles et les repositories interrogent la base. Postman sert d'interface de démonstration ; le sujet autorise une API et ne demande pas de page web [1, 3].

La collection `trips` conserve les colonnes TLC dans `data`, la provenance `source.file_id`/`source.row_number`, la durée `duration_s` et les flags. `zones` associe un identifiant entier aux noms et boroughs : les noms ne sont pas des clés uniques. `imports` conserve le SHA-256 du fichier, ses métadonnées et son état `loading`, `failed` ou `complete`. Les services ne sélectionnent que les imports complets. Aucun agrégat matérialisé n'est nécessaire aux services décrits ici.

Identité et reprise après intégration. Le prototype initial identifiait une ligne par SHA-256 du fichier et position. Le second membre a remplacé cette règle pour traiter les extraits chevauchants. L'identité actuelle combine l'heure de départ, une empreinte de toutes les colonnes canonisées et le rang parmi les lignes identiques du fichier. Le code utilise une empreinte Pandas de 64 bits, *distincte* du SHA-256 de provenance. Deux copies dans un fichier ont des rangs différents ; une ligne déjà importée retrouve son identité et l'index unique `_id` empêche sa réinsertion. L'import courant utilise des insertions par lots et traite les collisions de clé comme des lignes déjà présentes, pas les anciens upserts. Cette règle ne résout pas toute ambiguïté : deux enregistrements identiques issus de fichiers indépendants peuvent être confondus, une correction de contenu produit une nouvelle identité, et une collision d'empreinte reste possible. En cas de recouvrement, le document garde la provenance de la première insertion, pas toutes ses rééditions. La justification et la validation complète de F6 relèvent de la seconde contribution.

Dates et durées. Les heures locales TLC sont stockées en chaînes ISO sans fuseau. Les conversions UTC ne sont que temporaires pour calculer la durée : attribuer New York aux deux heures, convertir en UTC, puis soustraire. Une arrivée inexistante ou ambiguë produit `null`, pas une durée locale de secours. La ligne 942 762 de mars part le 9 mars à 23:38:43 et arrive le 10 à 03:40:14 : la durée réelle est 10 891 s, contre 14 491 s par soustraction locale. Elle porte `DST_SPANNING`. La ligne 841 170 arrive à 02:09:54 le 10 mars, heure qui n'existe pas : elle porte `DST_UNRESOLVED` et est exclue de journey.

Calculs côté serveur et traçabilité. `$match` filtre les documents ; `$project` prépare les champs utiles sans modifier la base ; `$group` regroupe et compte. `$sum:1` compte les candidats. Pour les retenus, `$cond` vaut 1 si la ligne est éligible et 0 sinon, puis `$sum` additionne ces valeurs. La même sélection produit ainsi les effectifs bruts et retenus. Le booléen est calculé par l'intersection entre les flags du trajet et ceux du profil, grâce à `$setIntersection`, `$size` et `$eq`. Une ligne cumulant plusieurs motifs n'est donc exclue qu'une fois.

Les groupes incluent le fichier source pour justifier les comptes. La version intégrée ajoute `GET /api/trips` : elle compte et liste, avec pagination, les lignes sous-jacentes selon zones, dates, créneau, profil et statut, et renvoie le fichier et la position d'origine. Ce complément ne faisait pas partie du premier prototype F1–F5. La présence de fichiers complets ne prouve pas la couverture de chaque heure ; les services ne fournissent pas non plus un instantané transactionnel entre la lecture des imports et l'agrégation.

4 Services analytiques F1 à F5

Les paramètres sont fournis à chaque appel ; les réponses JSON sont calculées sur les données stockées et renvoient les effectifs et sources. Les exemples numériques ci-dessous proviennent des réponses archivées du prototype sur janvier complet, à l'exception du contrôle DST sur des lignes de mars [5]. Ils ne sont pas des résultats recalculés sur les six mois de la version intégrée.

4.1 F1 : activité au départ ou à l'arrivée

`GET /api/activity` reçoit une zone, `pickup` ou `dropoff` et un intervalle. Il renvoie les comptes avant/après qualité, les répartitions par heure, jour de semaine, date et source. Le filtre utilise la zone et l'horodatage du sens demandé : une arrivée de février peut provenir d'un départ de janvier. Après `$match` et `$project`, `$facet` calcule les différentes répartitions à partir des mêmes candidats ; chaque branche utilise `$group` puis `$sort`. Les comptes par motif passent par `$unwind`, qui produit une ligne par flag. Ils peuvent se chevaucher. Pour les départs de la zone 181 en janvier, le contrôle Parquet archivé donne 983 candidats, 977 retenus et six exclus avec activity. Les distributions sont cumulées : ce ne sont pas des moyennes corrigées du nombre de lundis ou d'heures.

4.2 F2 : flux et directions opposées

`GET /api/flows` accepte une zone, un ensemble de zones ou un borough et une période. Il classe les destinations et paires, compare les retours et expose les séries hebdomadaires. Le filtre `$or` retient les départs *ou* arrivées dans l'aire pour disposer des deux sens, sans doubler les trajets internes. Les groupes MongoDB sont définis par origine, destination, semaine et source. Java combine les petits comptes pour les parts, séries et tris.

Les seuils par défaut demandent 20 trajets pour afficher un lien. Un lien régulier atteint cinq trajets dans au moins 75 % des semaines complètes ; sinon, une semaine représentant au moins 50 % du volume de référence peut le qualifier de pic. Moins de trois semaines complètes conduit à un historique insuffisant. Les seuils sont modifiables et les semaines partielles ne servent pas à cette classification. En janvier, Brooklyn représente 25 258 candidats et 24 860 départs retenus par flows. La paire 181 → 89 compte 48 trajets retenus, contre 18 en sens inverse ; elle est classée régulière. Le petit retour n'est pas transformé en flux majeur par une simple variation relative.

4.3 F3 : activité inhabituelle et couverture

`GET /api/unusual-activity` compare une zone/date/créneau aux mêmes jours de semaine d'une période antérieure explicite. Il expose les jours retenus, la médiane m, la dispersion MAD = mediane $|x_i - m|$ et les écarts. Les défauts demandent trois références, $m \geq 20$, un écart absolu au moins égal à 20, un écart relatif d'au moins 50 % et un score $|x - m|/(1{,}4826\,\text{MAD}) \geq 3$. Si MAD vaut zéro, le score est absent, un avertissement est ajouté et les critères absolu/relatif restent applicables. C'est un critère descriptif, pas une p-value.

Les jours fériés fédéraux 2024 et les journées entières de changement d'heure sont exclus des comparaisons usuelles. Le calendrier est embarqué avec sa source OPM [6]. Un second pipeline compte la ville par jour et créneau : une absence de mois source, de lignes ville, ou une forte variation générale peut limiter l'interprétation d'une baisse locale. Ce diagnostic ne prouve pas une lacune de collecte. Le prototype signale, par exemple, 55 départs retenus pour la zone 181 le 23 janvier contre une médiane de 20 sur ses références. La faible profondeur de cet historique et les jours exclus doivent accompagner cette hausse, sans en déduire une cause.

L'intégration a optimisé le pipeline ville : sur une journée entière, il ne calcule plus l'heure ; sur un créneau, il extrait directement les deux caractères de l'heure ISO avec `$substrBytes` et `$toInt`. Le délai maximal des agrégations F3 est passé à 300 s. Les temps initiaux ne mesurent pas ces optimisations.

4.4 F4 : caractéristiques des trajets

`GET /api/journeys` reçoit deux zones et deux périodes non chevauchantes. Il renvoie médiane, quartiles, p90, moyenne, écart-type, min/max et effectifs pour la durée et la distance. Les moins fiables sont décrits séparément ; les montants négatifs et doublons restent hors des statistiques. Les valeurs hors bornes $[Q_1 - 1{,}5\,IQR, Q_3 + 1{,}5\,IQR]$ sont comptées mais pas supprimées à nouveau.

MongoDB filtre la paire et projette durée stockée, distance, calendrier, source et flags. Un curseur fournit les lignes par lots de 2 000. Java calcule les quantiles exacts par interpolation à l'indice $(n-1)p$. Le choix évite un quantile approximatif ou un grand tableau BSON, au prix d'un transfert et d'une mémoire proportionnels aux candidats. La limite est de 200 000 lignes, avec erreur 422 explicite au-delà.

Pour tenir compte de trajets différents, une seconde comparaison utilise les strates semaine/week-end, créneaux de six heures et classes de distance. Les moyennes de durée des strates suffisamment représentées sont pondérées par les mêmes poids de référence ; les fractions couvertes sont affichées. Ce n'est pas une médiane standardisée ni une élimination de tous les facteurs de confusion. Pour 132 → 161, la médiane passe de 2 678 à 2 719 s entre les 1–14 et 15–31 janvier, avec 1 070 et 1 359 retenus. L'écart médian est 41 s ; la différence des moyennes standardisées vaut 49,76 s. Ces indicateurs ne suffisent pas à conclure à davantage de congestion.

4.5 F5 : changement de part face à la variation habituelle

`GET /api/flow-changes` compare des périodes de même durée, lundi à lundi, et utilise une période historique antérieure au courant. Le dénominateur N est l'ensemble des départs retenus de l'aire sélectionnée, pas seulement les flux du classement. Pour un flux, $p_A = n_A/N_A$ et $p_B = n_B/N_B$. La différence $p_B - p_A$ est comparée au p90 des variations absolues entre semaines historiques adjacentes, propres à ce flux. Le ratio est

$$R = \frac{|p_B - p_A|}{\max\{q_{0,9}(|p_w - p_{w-1}|),\, \varepsilon\}}.$$

Les défauts demandent 20 trajets dans au moins une période, 20 dans l'historique, trois semaines utilisables, un dénominateur hebdomadaire d'au moins 100, $\varepsilon = 0{,}001$ et $R \geq 2$. Les flux retenus sont classés par écart absolu de part, puis par identifiants ; les autres portent leur motif de non-qualification. Un dénominateur nul donne une part absente, pas un zéro concluant. Le plancher empêche un score infini sur un historique constant ; il ne rend pas le résultat statistiquement significatif.

La décomposition $n_B - n_A = (p_A N_B - n_A) + (n_B - p_A N_B)$ distingue l'effet du volume global et le résidu du flux. Entre les semaines du 15 et du 22 janvier, les départs retenus de la zone 132 passent de 30 263 à 26 739. Pour 132 → 123, les comptes passent pourtant de 29 à 146, soit de 0,0958 % à 0,5460 % : +0,4502 point. L'attendu au seul effet volume est 25,62 ; le résidu est +120,38. La baseline du 1er au 22 janvier ne fournit que deux différences hebdomadaires, une limite importante. L'intégration construit désormais les preuves par source après le classement, seulement pour les flux renvoyés, et porte le délai maximal de l'agrégation à 300 s, sans changer ce principe de calcul.

5 Mesures locales et validation

5.1 Choix d'index et alternative mesurée

Le choix comparé est une base avec le seul index automatique `_id`, puis une base avec des index adaptés aux filtres. Les deux index F1 associent respectivement zone/date de départ et zone/date d'arrivée. L'index F4 associe origine, destination et date de départ : les deux égalités précèdent l'intervalle. Ils ne remplacent pas la lecture des flags et métriques dans les documents.

Les essais archivés du 5 novembre utilisent janvier complet, 2 964 624 lignes, MongoDB 8.0.16 et Spring Boot 3.5.7 / Temurin 21.0.9 sur un serveur Linux. Le cgroup est limité à deux CPU et 8 Gio ; API et MongoDB partagent CPU, mémoire et disque. Pour chaque requête et configuration : un échauffement, puis trois appels mesurés, sans purge des caches. Les médianes ci-dessous excluent l'échauffement ; les temps individuels, écarts-types, réponses et plans `explain("executionStats")` sont archivés [5].

Service	Requête	Sans index	Index F1	F1 + F4
F2	Zone 181, janvier	2,313	0,209	—
F2	Brooklyn, janvier	3,464	1,330	—
F4	132 → 161	2,816	0,314	0,045
F4	161 → 236	2,515	0,433	0,186
F5	Zone 132	3,653	0,971	1,046
F5	Brooklyn	3,799	0,572	0,297

Les temps sont en secondes. F4 compare les 1–14 et 15–31 janvier ; F5 compare les semaines du 15 et du 22 janvier avec la baseline du 1er au 22. Les deux requêtes de F2 et les deux paires F4 satisfont une comparaison de conception sur plusieurs accès représentatifs ; elles ne remplacent pas E1–E3.

Pour F2 zone 181, les documents examinés passent de 2 964 624 à 7 924. Pour F4 aéroport, ils passent de 111 942 avec F1 à 2 459 avec F4. L'index de paire apporte donc un gain sur cet accès, au coût d'environ 54,1 Mio d'index supplémentaires et 7,46 s de construction dans cet essai. Le surcoût pendant l'import n'y est pas mesuré. Pour F5, F1 et F1+F4 examinent les mêmes 133 084 documents sur la zone 132 : les temps seuls n'établissent pas un bénéfice causal de F4. Aucun index F5 supplémentaire n'a été retenu.

Dans le premier F3, le contrôle ville restait un scan complet : les médianes 16,30/14,18 s pour la zone 132 sans/avec F1 montrent le gain limité du filtre local lorsque le reste domine. La version intégrée ajoute l'index temporel `pickup_datetime_zone`, nécessaire au placement par période, et optimise l'extraction de l'heure. Ces changements n'ont pas de mesure séparée dans les archives citées. `urbanflow.deploy` crée désormais les quatre index avant l'import sur les déploiements ; les anciens scripts manuels ne décrivent plus la procédure actuelle. Les premières observations Postman F1 sur le PC sont conservées, mais leur cache, effectif exact et variabilité ne permettent pas de les traiter comme une expérience contrôlée.

5.2 Résultats vérifiés et portée des preuves

Les archives de chaque benchmark enregistrent des réponses JSON identiques avant et après index. Les scripts indépendants relisent les Parquet sans utiliser les comptes MongoDB : F2/F5 vérifient flux et parts ; F3/F4 vérifient activité, statistiques et durées. Cela répond au principe de double vérification demandé par le sujet, avec au moins un résultat F2/F5 et un résultat F3/F4. Le contrôle DST de F4 porte sur de vraies lignes de mars dans une base temporaire ; il ne valide pas la couverture du mois entier.

Ces preuves concernent la version et les paramètres mesurés. Les scripts de vérification ont ensuite été adaptés à l'ingestion intégrée ; leur présence ne prouve pas leur réussite sur les nouveaux déploiements. Les mesures finales, l'égalité automatisée local/distribué, la mise à jour F6 et le scénario de panne appartiennent à la contribution expérimentale du second membre. Les scripts existent dans le dépôt ; les résultats correspondants ne sont pas remplacés par les essais locaux présentés ici.

Les seuils statistiques restent descriptifs, les calendriers sont incomplets pour les événements locaux et les flags ne détectent pas toute anomalie. Les comptages, historiques et taux d'exclusion doivent donc accompagner les résultats. Enfin, les essais sur un mois et trois répétitions ne prédisent ni le comportement sur six mois ni celui de plusieurs instances partageant une machine.

Références

[1] H. Akid, *Project 3 — UrbanFlow, Big Data and NoSQL*, M2 SLCD, 2026–2027. Fichier fourni : `Project3_UrbanFlow.pdf`, sections 2–6.

[2] NYC TLC, *Trip Record Data* et référentiel officiel. URLs et dates de récupération consignées dans `data/manifest.json`. `https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page`.

[3] UrbanFlow, état du code intégré `5f11cfd`, 7 novembre 2026 : `urbanflow/ingest.py`, `urbanflow/deploy.py`, `backend/src/main/java/fr/urbanflow/`. `https://github.com/deliasTheo/UrbanFlow-noSQL`.

[4] UrbanFlow, manifeste, rapport qualité et bilans d'import : `data/manifest.json`, `results/quality/quality_report.json`, `results/ingestion/2024-01.json` à `2024-03.json`.

[5] UrbanFlow, mesures initiales du 5 novembre 2026 : `results/performance/f1-indexes.md` et répertoires `results/performance/f2/` à `f5/`, notamment `measurements.json`, réponses, plans et vérifications.

[6] U.S. Office of Personnel Management, *2024 Federal Holidays*. Calendrier repris dans `backend/src/main/java/fr/urbanflow/unusual/Calendar2024.java`. `https://www.opm.gov/policy-data-oversight/pay-leave/federal-holidays/`.

A Provenance du jeu initial

Le manifeste est `data/manifest.json`. Les fichiers ci-dessous ont été récupérés le 2 novembre 2026 selon les métadonnées enregistrées. Les tailles, empreintes et effectifs ont été revérifiés lors de la préparation de ces annexes. Les numéros de ligne des exemples sont en base zéro.

Les URL Parquet suivent `https://d37ci6vzurychx.cloudfront.net/trip-data/` et les noms mensuels ci-dessous. Le référentiel vient de `https://d37ci6vzurychx.cloudfront.net/misc/taxi_zone_lookup.csv`.

`yellow_tripdata_2024-01.parquet` 2 964 624 lignes de données ; 49 961 641 octets.

SHA-256 : `c4d59da7bbc8abaeeeb1727947ee93d9891a71acb42854bd80db1571b2030510`.

`yellow_tripdata_2024-02.parquet` 3 007 526 lignes de données ; 50 349 284 octets.

SHA-256 : `c76c43c18c6c6664080dd920baab4928988d5786a6b65980792ca7cd796f9f20`.

`yellow_tripdata_2024-03.parquet` 3 582 628 lignes de données ; 60 078 280 octets.

SHA-256 : `2d4cdc8fb96726cdd3803b13b02d2e61e71d45720aff0ebc693a8bdd1f249823`.

`taxi_zone_lookup.csv` 265 lignes de données ; 12 331 octets.

SHA-256 : `1a99e105092230f8620f301edcca7f80d3080642ff404d28ed957d3fa222c8ed`.

Le rapport qualité couvre aussi avril–juin ; les effectifs utilisés ici sont ceux des trois mois initiaux. Les empreintes de tous les fichiers, y compris ceux réservés à F6 et E1–E3, restent dans le manifeste.

B Anomalies présentes et lignes justificatives

Fréquences issues de `results/quality/quality_report.json`, calculées *par fichier entier*, indépendamment des profils. Elles peuvent se recouper. Les huit exemples ont été comparés aux colonnes originales des Parquet et leurs durées recalculées avec New York puis UTC. Le relevé de cette vérification est `report/evidence-verification.json`. Une transition DST est une particularité à traiter, pas nécessairement une erreur de données.

Mois	Indicateur	Fréquence	Part du fichier (%)
2024-01	`PICKUP_OUTSIDE_FILE_MONTH`	18	0,000607
2024-01	`NON_POSITIVE_DURATION`	870	0,029346
2024-01	`ZERO_DISTANCE`	60 371	2,036380
2024-01	`NEGATIVE_AMOUNT`	35 504	1,197589
2024-01	`UNKNOWN_ZONE`	31 527	1,063440
2024-02	`EXACT_DUPLICATE`	1	0,000033
2024-03	`DST_SPANNING`	1 153	0,032183
2024-03	`DST_UNRESOLVED`	3	0,000084

2024-01, ligne 256 — `PICKUP_OUTSIDE_FILE_MONTH` Départ : 2023-12-31 23:56:46 ; arrivée : 2024-01-01 00:12:06. Zones 236 → 142, fournisseur 2, distance 2.38 miles, total 21.6. Le départ est en décembre alors que la ligne est publiée en janvier ; la requête doit utiliser la date réelle.

2024-01, ligne 87 994 — `NON_POSITIVE_DURATION` Départ : 2024-01-02 10:00:00 ; arrivée : 2024-01-02 09:53:56. Zones 75 → 95, fournisseur 1, distance 7.2 miles, total 46.94. L'arrivée précède le départ : durée réelle −364 s, exclue du profil journey.

2024-01, ligne 17 — `ZERO_DISTANCE` Départ : 2024-01-01 00:52:09 ; arrivée : 2024-01-01 00:52:28. Zones 237 → 237, fournisseur 2, distance 0.0 miles, total 8.0. Distance nulle malgré 19 s entre les deux heures : la ligne reste stockée mais n'alimente pas journey.

2024-01, ligne 99 — `NEGATIVE_AMOUNT` Départ : 2024-01-01 00:18:24 ; arrivée : 2024-01-01 00:30:39. Zones 249 → 232, fournisseur 2, distance 2.16 miles, total -18.5. Le montant total négatif est exclu des profils analytiques. Le rapport qualité repère aussi cette ligne comme écriture d'annulation.

2024-01, ligne 24 — `UNKNOWN_ZONE` Départ : 2024-01-01 00:42:05 ; arrivée : 2024-01-01 01:16:49. Zones 263 → 265, fournisseur 1, distance 23.9 miles, total 127.94. L'arrivée 265 ne désigne pas une zone précise de New York : exclusion du profil flows et donc journey.

2024-02, ligne 754 252 — `EXACT_DUPLICATE` Départ : 2024-02-08 18:23:50 ; arrivée : 2024-02-08 18:38:14. Zones 162 → 230, fournisseur 2, distance 0.81 miles, total 23.16. Le rapport qualité signale cette ligne comme copie d'une ligne précédente du même fichier ; seule la copie est exclue. Les champs cités identifient l'exemple, le rapport JSON conserve toutes les colonnes. La ligne identique antérieure a été retrouvée à la position 754 248.

2024-03, ligne 942 762 — `DST_SPANNING` Départ : 2024-03-09 23:38:43 ; arrivée : 2024-03-10 03:40:14. Zones 249 → 203, fournisseur 2, distance 31.35 miles, total 131.14. Durée UTC de 10 891 s contre 14 491 s locales : une heure est retranchée. Le flag n'exclut pas automatiquement le trajet de journey.

2024-03, ligne 841 170 — `DST_UNRESOLVED` Départ : 2024-03-09 02:31:02 ; arrivée : 2024-03-10 02:09:54. Zones 114 → 48, fournisseur 2, distance 3.36 miles, total 27.72. L'arrivée est à une heure inexistante le 10 mars. La durée exploitable est null et non les 85 132 s de différence locale affichées par l'ancien rapport de diagnostic.